exercise 6.

Problem 1

1. write the general class for this object.

The class represents a tree node with a weight attribute and a list of children. Each node stores its own weight value and references to its child nodes.

```
class WeightedTreeNode:
    def __init__(self, weight, children=None):
        self.weight = weight
        self.children = children if children is not None else []
    def __repr__(self):
        return f"WeightedTreeNode(weight={self.weight:.6f},
num_children={len(self.children)})"
```

A tree node where each node has n children, each child's weight = parent's weight / n.

Attributes : weight (float): The weight of this node
children (list): List of child WeightedTreeNode objects

2. Generate a tree of depth N=3, with initial parent tree of weight 1/n.

The tree is generated recursively. The root node has weight 1/n. Each node at the current level creates n children, each with weight equal to the parent's weight divided by n. The recursion stops when depth reaches 1 (leaf level)

At level k (0-indexed), there are n^k nodes, each with weight $(1/n)^{k+1}$. The sum at each level is $n^k \times (1/n)^{k+1} = 1/n$.

With depth = n (eg: n levels), the total sum $= n \times (1/n) = 1$.

Therefore, when depth equals n, the total weight always sums to 1 regardless of the value of n.

Args : n (int) : Number of children per node
depth (int) : Number of levels in the tree (depth = n ensures total weight sums to 1)
weight (float): Weight of the root node (default: 1/n)

Returns : WeightedTreeNode : The root of the generated tree.

```
def generate_tree (n, depth, weight = None):
    if weight is None:
        weight = 1.0/n
    node = WeightedTreeNode (weight) # if depth >1, create n
                                       children recursively.
    if depth >1:
        child_weight = weight /n
        for _ in range (n):
            child = generate_tree (n, depth -1, child_weight)
            node.children.append(child)
    return node.
# example : n=3, depth=3
tree = generate_tree (n=3, depth =3)
```

3.

Depth-first recursive traversal that sum all node weights. The DFS function recursively visits each node, starting from the root. It adds the current node's weight to the running total, then recurses on each child. This naturally follows the call stack's LIFO order, fully exploring one subtree before moving to the next

Args: node (WeightedTreeNode): Current node.

Returns: float: sum of all weights in the subtree.

```
def dfs_sum (node):
    total = node.weight # start with current node's weight
    for child in node.children: # Recursively add children's weights
        total += dfs_sum (child)
    return total.
```

4. Same with breadth first, check also 1!

Breadth-first traversal that sum all node weights using a queue. BFS uses a FIFO queue to process nodes level by level. Starting with the root, each node is dequeued, its weight is added to the total, and its children are enqueued. This guarantees nodes are visited in breadth-first order.

```
def bfs_sum(node):
    total = 0.0
    queue = deque([node])
    while queue:
        current = queue.popleft()
        total += current.weight
        for child in current.children: #Add all children to the queue
            queue.append(child)
    return total
```

Args: node (WeightedTreeNode): Root node.

Returns: float: sum of all weights in the tree.

5. return 1 and -1

After visiting a node and adding its weight to the sum, we flip the sign of the weight (multiply by -1). After the first traversal (DFS), all weights become negative, so the sum = 1 (original positive weights). After the second traversal (BFS), all weights flip back to positive, so the sum = -1 (was summing negative weights).

```
def dfs_sum_flip(node):
    total = node.weight          # Add current weight
    node.weight = -node.weight   # Flip sign: pos → neg
    for child in node.children:
        total += dfs_sum_flip(child)  # Recurse on children
    return total

def bfs_sum_flip(node):
    total = 0.0
```

```
queue = deque([node])
while queue:
    current = queue.popleft()
    total += current.weight  # Add current weight
    current.weight = -current.weight  # Flip sign: neg→pos
    for child in current.children:
        queue.append(child)
return total
```

DFS that sums weights and flips each node's sign after visiting.

Visit: node → add weight to sum → flip sign → recurse on children

Summarize.

After first call: all weights become negative, returns 1 (original positive sum).

After second call: all weights become positive again, returns -1 (negative sum).

6. Write recursive and non recursive versions of breadth first.

Non-recursive (iterative) BFS using a queue - standard implementation. This is the standard BFS using a FIFO queue.

```
def bfs_non_recursive(node):
    result = []
    total = 0.0
    queue = deque([node])
    while queue:
        current = queue.popleft()  # Dequeue front
        result.append(current.weight)
        total += current.weight
        for child in current.children:  # Enqueue children
            queue.append(child)
    return total, result
```

Recursive BFS using level-by level processing.
Since BFS naturally requires a queue (FIFO), the recursive version processes one level at a time, passing the next's nodes recursively.

```
def bfs_recursive(node):
    def_bfs_level(current_level):
        if not current_level:  # Base case: no more nodes
            return 0.0
        total = 0.0
        next_level = []
        for node in current_level:  # Process all nodes at level
            total += node.weight
            next_level.extend(node.children) # collect next level
        total += bfs_level(next_level) # Recurse on next level
        return total
    return _bfs_level([node])  # start with root.
```

Both versions produce identical results.

7.

While DFS naturally maps to recursion (the call stack acts as the implicit stack needed for DFS), BFS requires a FIFO queue which does Not map naturally to the call stack.

(1) BFS processes nodes level by level, requiring a queue (FIFO order). The call stack provides LIFO (Last-in-First-Out) order, which naturally suits DFS, not BFS.

(2) A recursive BFS must collect all nodes at the next level before making the recursive call. This means we must build an explicit list of next-level nodes anyway, which defeats the purpose of using recursion.

(3) Space complexity: Recursive BFS stores all nodes of the widest level in memory as function arguments. For a tree with n children and depth d, the widest level has n^d nodes, which can cause stack overflow for large trees.

4) the recursive version adds overhead (function call overhead per level) without gaining any conceptual simplicity. The iterative queue-based BFS is both more natural and more efficient. In contrast, DFS maps perfectly to recursion because the call stack naturally provides the LIFO behavior needed to explore one branch fully before backtracking.

Testing.

```
def run_tests()
    print("="*70)
    print("exercise 6")

    test_values = [2,3,4,5] # Test for various n
    all_passed = True # Track overall result
Q2  Q3
    print("\n[Q2, Q3] DFS Recursive Sum")
    print(f"{'n':<4} {'depth':<6} {'DFS Sum':<20} {'Result'}")
    print("  " + "-"*44)
    for n in test_values:
        tree = generate_tree(n, depth=n) # Generate tree
        result = dfs_sum(tree)
        passed = abs(result - 1.0) < 1e-9
        all_passed &= passed
        print(f"{n:<4} {n:<6} {result:<20.10f} {'PASS' if
            passed else 'FAIL'}")
Q4
    print(f"\n[Q4] BFS Iterative Sum")
    print(f"{'n':<4} {'depth':<6} {'BFS Sum':<20} {'Result'}")
    print("  " + "-"*44)
    for n in test_values:
        tree = generate_tree(n, depth=n)
        result = bfs_sum(tree)
        passed = abs(result - 1.0) < 1e-9
        all_passed &= passed
        print(f"{n:<4} {n:<6} {result:<20.10f} {'PASS' if
            passed else 'FAIL'}")
```

Q5

```
print (f"\n[Q5] DFS/BFS With Sign Flipping")
print (f" {'n':<4} {'1st (DFS)':<16}
        {'2nd (BFS)':<16} {'Result'}")
print (" " + "-" *48)
for n in test_values:
    tree = generate_tree (n, depth=n)
    first = dfs_sum_flip (tree) # 1st: DFS flip → expect 1
    second = bfs_sum_flip (tree) # 2nd: BFS flip → expect -1
    passed = abs (first - 1.0) < 1e-9 and abs (second + 1.0) < 1e-9
    all passed e = passed
    print (f" {n:<4} {first:<16.10f} {second:<16.10f}
        {'PASS' if passed else 'FAIL'}")
```

Q6.

```
print (f"\n[Q6] Recursive vs Non-Recursive BFS")
print (f" {'n':<4} {'Non-Recursive':<20} {'Recursive':<20}
    {'Result'}")
print (" " + "-" *52)
for n in test_values:
    tree = generate_tree (n, depth=n)
    nr_total = bfs_non_recursive (tree) # Iterative BFS
    r_total = bfs_recursive (tree) # Recursive BFS
    passed = abs (nr_total - r_total) < 1e-9
    all passed e = passed
    print (f" {n:<4} {nr_total:<20.10f} {r_total:<20.10f}
        {'PASS' if passed else 'FAIL'}")
if __name__ == "__main__":
    run_tests()
```